FOL: Bridging Object-Oriented and Functional Programming via the Metaobject Protocol

Frank Adrian
Ancar Technology
Olathe, KS, USA
frank.adrian314159@gmail.com

Abstract

FOL (Functional Object Lisp) provides CLOS-based object orientation backed by persistent data structures, using the Common Lisp Metaobject Protocol (MOP) to control object access and mutation. We contribute: (1) a persistent metaclass system with hybrid storage and lazy schema migration, and (2) a specificity-ordered predicate dispatch model.

Benchmarks show only a 1.4–2.1× read overhead for persistent slot access. Though updates can take 33–447× the overhead of Common Lisp's slot mutation, the per-operation cost is a fixed $O(\log n)$ structural copy; this is amortized over growing workloads, so whole-program ratios narrow significantly at scale. In an AST optimization engine with 25 node classes, 54 predicate-dispatch rewrite rules, and a 9,999-node balanced tree, FOL is 47× slower per optimizer pass than a `defstruct+typecase` baseline while providing $O(1)$ speculative rollback. Whole-program overhead in a logic simulation comprising 1.18-billion-gate evaluations narrows to 6.9× execution time when gate-connectivity is held constant and only the priority queue is persistent.

Our predicate dispatch introduces a syntactically computable, level-based ordering that provides a predictable alternative to logical subsumption; closed `defn` dispatch matches hand-written `cond` cascade performance, and load-time method sorting runs in $O(N \log N)$ versus at least $O(N^2)$ pairwise subsumption checks per generic function required by theorem-prover-based systems.

CCS Concepts

• **Software and its engineering → Functional languages**; **Object oriented languages**; *Extensible languages.*

Keywords

Functional programming, object-oriented programming, Lisp, persistent data structures, CLOS, MOP

Common Lisp's object system (CLOS) offers powerful extensibility via the MOP, but objects are inherently mutable: concurrent speculative updates require synchronization or explicit pre-call copies, and schema evolution requires hand-written migration methods. FOL (Functional Object Lisp[1]) addresses these limitations by leveraging the MOP [16] to back every class instance with persistent data structures, providing intrinsic immutability, lazy schema migration, and a novel predicate dispatch system. The CLOS programming model is preserved for reads—generic function calls, method dispatch syntax, and `slot-value` for reading all behave as in standard CLOS—with slot mutation replaced by functional `assoc`/`dissoc` rather than `setf slot-value`. The novel aspects of the metaclass contribution are: a hybrid native/trie threshold strategy calibrated to the empirical class-width distribution [17]; lazy alias-based schema migration via multi-hop chain replay that recovers renamed slots automatically without hand-written migration methods; and the integration of specificity-ordered predicate dispatch with standard CLOS method combinations. FOL represents objects as versioned points in time, where "mutation" is reified as a transition between immutable snapshots.

FOL eases the "new operations" dimension of the Expression Problem: adding a traversal such as `count-ops`—which recurses over `:left`/`:right` slots and returns a value at leaves—requires only a new `defmethod` with one clause per handled node type; no existing class definitions change, and after rolling back a failed optimization pass, `count-ops` applies to the reverted tree without copying. Like CLOS, the "new data types" dimension is not fully solved: adding a new node type still requires a new clause in each generic that must handle it. However, schema evolution—the most common structural change in practice—is automatic: slot renames and additions are recovered lazily without hand-written migration methods (Section 3.2), and all existing clauses remain correct over any mix of original and speculatively-modified nodes.

Following Hickey's epochal time model [14], FOL distinguishes *identity* (a named entity whose state changes over time) from *value* (an immutable snapshot at a point in time). In FOL, read access to values is always lock-free, as threads observe stable snapshots; coordinated state mutation is handled via Atom (CAS-based) and Ref (STM) primitives.

1 Architecture

1.1 Object Implementation

Slot storage uses a hybrid strategy: objects with ≤ 8 slots use native CLOS slots, while wider objects overflow into a 32-way persistent trie. This threshold keeps ≈80% of classes [17] on the native fast path; Table 6 shows $T = 8$ also minimizes modeled update cost for the 8-slot case, meeting both objectives. All redefinitions are handled lazily at the next object mutation (`assoc`, etc.).

Immutability is enforced via the `persistent-class` metaclass, which intercepts slot access (Table 1) to: (1) guard against mutation; (2) map logical slots to the hybrid layout; and (3) transform instances for schema evolution. A reserved native slot `%metadata` stores the object's metadata map; it is excluded from structural equality via `=`—attaching a `:last-accessed` timestamp does not alter its identity as a key in a persistent map or invalidate a prior hash-join result. Objects differing only in metadata are `=` and hash identically; `call-next-method` chain behaviour is unaffected by metadata state.

[1] *Functional*: persistent value semantics; *Object*: MOP-integrated class instances.

Table 1: MOP Protocol Adaptation

Protocol	Status	Rationale
`slot-value-using-class`	Redirected	Reads from base object or overflow trie
`(setf slot-value-using-class)`	Guarded	Blocks post-init writes
`slot-boundp-using-class`	Redirected	Checks object slot membership
`slot-makunbound-using-class`	Blocked	Immutability invariant — use `dissoc` to unbind slots
`validate-superclass`	Extended	Cross-metaclass mixing
`initialize-instance`	Extended	Populates base object and overflow trie from initargs; stamps `%schema-version` at birth
`reinitialize-instance`	Extended (`:before` on metaclass)	Snapshots current alias-map, overflow indices, and version counter into a `class-version` struct before each redefinition; advances the version counter
`compute-slots`	Specialized	Implements hybrid native/trie layout
`update-instance-for-redefined-class`	Implemented (`:after`)	Multi-hop chain replay: composes alias-maps from version chain back to birth version, recovering renamed values (Sec. 3.2)
`change-class`	Omitted	Changing a live object's class violates immutability; callers must construct a new instance of the target class instead

Method combination uses standard CLOS execution semantics (`:before`/`:after`/`:around`), with method ordering determined by $\prec_{disp}$ in place of the class precedence list.

1.2 Collections and Runtime

FOL implements persistent **Vectors** (32-way tries) [19], **Maps/Sets** (32-way HAMTs) [2], and **Sorted collections** (32-way B-trees). The branching factor of 32 ensures depth ≤ 5 for million-element collections, bounding path-copying costs. For keyword keys, HAMT lookup is $\approx$1.3× faster than standard CL `gethash` on SBCL,[2] as EQL-based key comparison avoids the general hashing path and index computation uses POPCNT.

FOL transpiles to Common Lisp, inheriting unwrapped primitives and generational GC performance.

1.3 Space Complexity

FOL persistent collections do not retain historical versions; nor do mutable identity types (Atoms, Refs, Agents), which hold only the current value—old snapshots become eligible for GC as soon as no live reference remains.[3] Structural sharing bounds the incremental cost of each `assoc` to $O(\log_{32} N)$ new nodes (one path copy), not $O(N)$. Short-lived intermediates—loop variables, traversal temporaries—are reclaimed in SBCL's nursery, keeping GC overhead at 2.0% at the 32×32×32-30000 LSim scale; smaller circuits incur proportionally higher GC pressure as the per-merge allocation cost is not yet amortized.

2 Features

FOL adopts Clojure's literal syntax and sequence APIs and provides: **Transducers** [15] decouple algorithm logic from collection type. **Lazy sequences** defer computation, enabling infinite sequences. **Atoms** provide CAS mutable references; **Refs** provide STM with optimistic concurrency and retry; **Agents** handle asynchronous state updates. **Transient builders** reduce allocation overhead for bulk updates by confining mutation to a thread-local window; W successive writes cost $O(W \log_{32} N)$ time but allocate only $O(\log_{32} N)$ nodes (one initial path copy). **Macros** follow Clojure-style expansion.

Generic functions dispatch by arity, then specificity. **Closed dispatch** (`defn`) emits a plain `defun` containing a `cond` cascade (full clause set known at load time); `call-next-method` is unavailable. **Open dispatch** (`defmethod`) supports CLOS-style extensibility; use it when `call-next-method` or auxiliary qualifiers (`:before`/`:after`/`:around`) are needed.

3 Predicate Dispatch

Figure 1 gives the formal grammar for FOL's multi-pattern dispatch syntax.

3.1 Dispatch Ordering Rules

FOL extends pattern matching with predicate specializers (`var (fn args...)`) that evaluate as `(fn var args...)` at dispatch time, gating the clause on a runtime condition. `(x (even?))` is more specific than `(x <integer>)`: it constrains value space beyond class membership, so FOL orders predicates above types. Specificity categories (Table 2) order patterns by constraint strength (Level 3 > 2 > 1 > 0), resolving same-level ties via definition order. This *syntactic precedence model* selects the narrowly specialized form (e.g., `even?` over `<integer>`) via a computable total order $\prec_{\text{disp}}$. Within each level, the ordering mirrors semantic subsumption: types via the

[2] Mean of 10^6 single-key lookups on a 1,000-entry map, SBCL 2.6.0 x86-64; reported as context for the collection design rather than a primary evaluation result.

[3] This concerns *automatic* history retention. User-managed versioning is trivially available: because each `assoc` returns a new root, a caller can hold onto any previous root to preserve that snapshot indefinitely—structural sharing ensures retained versions cost only $O(\log_{32} N)$ extra nodes each, not a full copy.

```
defn       ::=  (defn name (s-clause | m-clause)) | (fn (s-clause | m-clause))
defg/m     ::=  (defgeneric name...) | (defmethod name qualifier? (s-clause | m-clause))
s-clause   ::=  [param* ] body+
m-clause   ::=  {(symbol param | :keys [(symbol | (sym param))* ])* } body+
param      ::=  symbol | type | (symbol type) | (symbol (fn arg*)) | destr
destr      ::=  [param* ] | {:keys [(symbol | (sym param))* ]}
type       ::=  <type-name>
qualifier  ::=  :before | :after | :around
```

Figure 1: FOL Multi-Pattern Dispatch Grammar (compressed)

class hierarchy DAG, structural patterns via argument-arity containment, wildcards by definition. Cross-level priority (predicate over type over destructuring over wildcard) reflects operational structure: a type specializer establishes class membership, while a predicate specializer imposes a runtime value test the type system cannot express. The cost: a predicate covering a strict subset of one type (e.g., `even?` within `<integer>`) always fires before a broader type clause, occasionally requiring clause reordering. Level 3 (arbitrary predicates) is ordered by definition index only; the system detects no semantic subsumption (e.g., `prime?` vs `pos-int?`) within this stratum. Programmers cannot declaratively express subsumption between independent predicates; arbitrary overlaps must be resolved by load-order convention or encapsulated in a single clause. This deliberately trades theorem-prover exactness for an algorithmically bounded compilation phase. All method qualifiers (`:before`, `:after`, `:around`) follow this computed order. Standard CLOS execution semantics are preserved—e.g., `:before` methods run most-specific-first, `:after` run least-specific-first—but the sequence they iterate over is $\prec_{\text{disp}}$ instead of the class precedence list.

Table 2: Predicate Specificity Categories

Level	Category	Example
3	Predicates	`(x (even?))`
2	Types	`(x <integer>)`
1	Destructuring	`[x y]`
0	Wildcard	`x`

For compound predicates, the level calculus applies $\mathcal{L}(\text{and}\, p_1 \dots p_n) = \max_i \mathcal{L}(p_i)$: a conjunction of type predicates is Level 2 (static), while a mixed conjunction is Level 3 (dynamic). This is semantically conservative but sound. Disjunction and negation (`or`, `not`) are not supported as dispatch combinators; predicates requiring them must be encoded as a single Level-3 function. Conjunction is supported because its level is bounded by max: the result is at least as specific as its most dynamic component, giving a sound syntactic upper bound. A disjunction, by contrast, could match via either branch; determining which of two disjunctive patterns is more specific requires subsumption reasoning over all possible inputs—which requires a theorem prover.

One consequence: `(and (integer?) (prime?))` receives Level 3 ($\max(2, 3) = 3$), the same level as the bare predicate `(prime?)` alone. The conjunction is no less constrained, yet the purely syntactic calculus cannot determine semantic subsumption without a theorem prover, which FOL avoids. The mitigation is definition order: place the conjunction clause first. A future composite level for type-annotated predicates (e.g., `(x <integer> (prime?))`) could recover automatic ordering without a theorem prover.

3.1.1 Formal Specificity Ordering. We formalize the specificity relation $\prec$ using the inference rules summarized in Table 3. Let $\mathcal{L}(p) \in \{0, 1, 2, 3\}$ be the specificity level of pattern p.

Table 3: Specificity Inference Rules

Rule	Formalization
Spec-Level	$\dfrac{\mathcal{L}(p_1) > \mathcal{L}(p_2)}{p_1 \prec p_2}$
Spec-Type	$\dfrac{C_1 \sqsubset C_2}{(x\ C_1) \prec (x\ C_2)}$
Spec-Prefix	$\dfrac{p_1 \prec q_1}{[p_1, \dots] \prec [q_1, \dots]}$
Spec-Tail	$\dfrac{\mathcal{L}(p_1) = \mathcal{L}(q_1) \qquad [p_2, \dots] \prec [q_2, \dots]}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n]}$
Spec-Length	$\dfrac{\mathcal{L}(p_i) = \mathcal{L}(q_i)\ \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n, p_{n+1}] \prec [q_1, \dots, q_n]}$
Spec-Fixed	$\dfrac{\mathcal{L}(p_i) = \mathcal{L}(q_i)\ \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n\ \&\ r]}$
Spec-Map-Prefix	$\dfrac{p_1 \prec q_1}{\{k_1\ p_1, \dots\} \prec \{k_1\ q_1, \dots\}}$
Spec-Map-Tail	$\dfrac{p_1 = q_1 \qquad \{k_2 \dots\} \prec \{k_2 \dots\}}{\{k_1\ p_1, k_2\ p_2, \dots\} \prec \{k_1\ q_1, k_2\ q_2, \dots\}}$
Spec-Keys	$\dfrac{\text{keys}(q) \subsetneq \text{keys}(p)}{p \prec q}$

Spec-Type handles within-Level-2 ordering: $C_1 \sqsubset C_2$ denotes strict subclass in the CLOS class hierarchy DAG (i.e., C_1 is a proper subclass of C_2), delegating type subsumption to the existing CPL.

The operational dispatch order $\prec_{\text{disp}}$ resolves ties using definition order. Let $idx(p)$ denote the definition index of the method clause containing p.

$$p_1 \prec_{\text{disp}} p_2 \iff p_1 \prec p_2 \vee (\neg(p_2 \prec p_1) \wedge idx(p_1) < idx(p_2))$$

Patterns of different arity are incomparable; dispatch selects the arity group first, then applies $\prec_{\text{disp}}$, which is total on same-arity patterns (Lemma 1). Definition-order tie-breaking follows Lisp's interactive workflow.

Lemma 3.1 (Totality). *$\prec_{disp}$ is a strict total order on same-arity patterns.*

Proof. $\prec$ is irreflexive: Spec-Level requires $\mathcal{L}(p_1) > \mathcal{L}(p_2)$, which is false when $p_1 = p_2$; Spec-Prefix, Spec-Tail, Spec-Length, and Spec-Fixed each require a strict inequality at some element position, vacuous when all elements are equal; Spec-Map-Prefix and Spec-Map-Tail recurse to the same base case; Spec-Keys requires $\text{keys}(q) \subsetneq \text{keys}(p)$, which is false when $p = q$; Spec-Type requires $C_1 \sqsubset C_2$, which is false when $C_1 = C_2$. $\prec$ is also transitive: the rules encode a lexicographic comparison on each pattern's level vector, arity, and key set; transitivity follows by induction on pattern length, with Spec-Level, Spec-Prefix, and Spec-Tail each inheriting transitivity from their sub-comparisons, and Spec-Keys from set-containment transitivity. For any same-arity pair, either $\prec$ yields an ordering directly, or the pair is incomparable under $\prec$ (patterns differing only in variable names, or mixed sequential/map patterns); in the latter case every clause carries a unique load-time *idx*, so $\prec_{\text{disp}}$ is total. □

3.1.2 Dispatch Performance. Predicate dispatch requires an $O(N)$ linear scan per call, as Level-3 resolution cannot be statically cached. Methods are sorted once at load time in $O(N \log N)$ using the syntactic level order; theorem-prover-based systems instead require pairwise subsumption checks at load time (at least $O(N^2)$ per generic function) but produce faster cached dispatch. To mitigate runtime cost, FOL partitions methods by arity group, ensuring calls only scan matching-arity methods. `call-next-method` resolves the next method under $\prec_{\text{disp}}$, not the CLOS class precedence list; auxiliary sequences (`:before`/`:after`/`:around`) follow the same order, preserving CLOS execution semantics.

3.1.3 Transpiler Architecture. FOL transpiles to Common Lisp via four stages: **reader expansion** (literal syntax), **macro expansion** (Clojure-style macros to FOL primitives), **code generation** (MOP-integrated CL emission), and **linking**.

Listing 1 illustrates a two-clause `defn` with a type specializer. The compiler emits a direct `defun` containing a specificity-ordered `cond` cascade; no runtime dispatch infrastructure is invoked, as the ordering is resolved statically at compile time.

Listing 1: Transpilation: specificity-ordered multi-clause `defn`

```
;; FOL source: type specializer (Level 2) before wildcard (Level 0)
(defn classify
  [(x <integer>) :integer]
  [x             :other])

;; Generated Common Lisp: ordering preserved as cond cascade
;; (class references bound once via load-time-value)
(defun classify (x)
  (cond
    ((typep x (load-time-value (find-class '<integer>))) :integer)
    (t                                                   :other)))
```

Comparison to `defmulti`. Clojure's `defmulti` [13] dispatches on the return value of a user-supplied dispatch function, with ambiguity resolved manually via `prefer-method`. FOL's syntactic total order $\prec_{\text{disp}}$ eliminates this burden for patterns differing in type or structural depth; the residual hazard—incomparable Level-3 predicates, discussed in Section 5—has the same scope as `prefer-method` but is narrower because structural and type differences are resolved syntactically.

3.1.4 Usage Patterns. Three patterns illustrate the combination of CLOS features and predicate dispatch. **Structural Diff** (Listing 2) uses an `:around` method on `assoc` to detect changes by comparing snapshots. **Speculative Execution** (Listing 3) provides $O(1)$ rollback: returning the pre-update snapshot instead of the modified object, with no copying or undo log. **Content routing** (Listing 4) dispatches on data values, replacing manual `cond` cascades with declarative clauses.

Listing 2: Automatic Structural Diff

```
(defmethod assoc :around [(obj <diffable>) key val]
  (if (= key :_changes)
      (call-next-method)          ; :_changes key routes to primary,
       no re-entrance
      (bind [old (get obj key)
             new-obj (call-next-method)]
        (if (= old (get new-obj key))
            new-obj
            (assoc new-obj :_changes (inc (get obj :_changes 0)))))))
```

Listing 3: Speculative Execution Guard

```
(defmethod assoc :around [(obj <guarded>) key val]
  (if (valid-update? obj key val)
      (call-next-method)
      obj))  ; discard update by returning original
```

Listing 4: Value-Based Alerting

```
(defmethod notify [{:keys [(temp (>= 100))]}]
  (print "ALERT: Critical Temperature!"))
```

Predicate arguments are evaluated at compile time and closed over: `(>= 100)` partially applies `>=` to the literal `100`, producing a unary predicate `(lambda (temp) (>= temp 100))` that gates the clause at dispatch time.

3.2 Schema Evolution

The `:after` method on `update-instance-for-redefined-class` (*chain replay*) walks a *version chain* on the metaclass, composing single-hop alias maps from Birth to Current. This map is consulted against the `property-list` of discarded slots to recover renamed values, rebuilding the overflow trie if necessary. Chain replay fires lazily at first post-redefinition access, so all intermediate hops since the object was last touched are composed in a single pass before any mutation is applied. This migration is confluent: alias maps compose as name-remapping functions, and macro-expansion ensures slot names are disjoint across hops (no slot is both source and target), so replaying hops in any order yields the same result. The macro-expansion step also detects cyclical renames ($A \to B \to A$) and fan-in conflicts (both X and Y renamed to Z), signaling a compile-time error before redefinition.

Listing 5: Lazy Schema Migration with Aliasing

```
;; Schema 1
(defclass <sensor> [] [[id] [reading]])
;; Schema 2: rename reading -> val; :reading keyword still routes to
      val
```

```
(defclass <sensor> [] [[id] [val :alias reading]])
```

The `:alias` annotation stores `:reading` in the slot's alias map; at first post-redefinition access, chain replay consults the property-list of discarded slots, finds `:reading`, and writes its value into the new `:val` slot. After migration, `(get obj :reading)` also resolves via the alias map to `:val`, so existing call sites need no updates.

3.2.1 Persistence and Versioning Performance. Versioning overhead—the cost of recovering the current state of an object via chain-replay—is $O(H \cdot A)$ where H is the number of redefinitions and A is the average number of aliases per hop. Since redefinitions are infrequent and alias sets are small, $H \cdot A$ is typically < 50. Replay cost is bounded by a few microseconds, paid once per dormant instance on first access after redefinition. Once migrated, the object's slots reside in the current schema's preferred layout (native or overflowed), and subsequent reads proceed at the standard 1.4–2.1× read rate.

4 Evaluation

4.1 Common Lisp and Clojure Comparison

FOL fills a gap between CLOS and Clojure (Table 4): where `defrecord` provides map-backed values, FOL provides MOP-integrated persistence with full CLOS method combinations.

Table 4: Built-in Feature Comparison

Feature	FOL	Clojure	CL
Static type safety	×	×	×
Persistent objects	✓[d]	○[c]	×
CLOS/MOP	✓	×	✓
Method combinations	✓	×	✓
Predicate dispatch	✓	single[a]	×[b]
Schema evolution	✓[e]	×	Manual
Transducers	✓	✓	×
Macros	✓	✓	✓
Raw scalar performance	Low[f]	High	High

[a] `defmulti` dispatches via a user-supplied function (arbitrary arity); `prefer-method` and `derive`-based hierarchies allow manual ordering overrides, but there is no automatic multi-parameter specificity ordering derived from the syntactic form of the dispatch expression.
[b] Multi-parameter type dispatch natively; libraries like `trivia` add pattern matching.
[c] Clojure has persistent *data structures*; `defrecord` produces a map-backed value type, not a MOP-integrated persistent object.
[d] Metaclass-integrated persistent class instances: slot storage is backed by immutable HAMTs, `(setf slot-value)` is blocked, and functional updates are performed via `assoc`. Persistence is enforced transparently by the metaclass protocol.
[e] Lazy alias-based chain migration: renamed slots recovered automatically at first post-redefinition access; multi-hop chains composed in a single pass.
[f] Scalar arithmetic and non-slot operations inherit full CL performance. "Low" refers specifically to persistent object slot access: reads add 1.4–2.1× overhead and writes add 33–447× overhead relative to mutable CLOS.

4.2 Micro-Benchmarks

We measured persistent objects vs. mutable CLOS across slot counts. **Construction** costs ≈183 ns for a 2-slot object vs. ≈37 ns (mutable `make-instance`, 5×) and ≈25 ns (`make-struct`, 7×). The `%make-persistent` optimisation bypasses `initialize-instance` via direct `slot-value` writes, delivering a 4.8× speedup (877 ns → 183 ns) and confirming that MOP initarg iteration dominates construction. **Reads** are 1.4–2.1× native; **Updates** are 33–447× slower (Table 5). Table 6 shows $T = 8$ is optimal for the 8-slot case and achieves the minimum for $N \geq 32$; it yields to higher thresholds only at intermediate widths ($N = 12$, $N = 16$) that are under-represented relative to the ≤8-slot majority [17].

Table 5: Actual slot update overhead (μs/op) vs. native CLOS

Slots	Persistent	Mutable	Ratio	Notes
2	1.14 μs	34.6 ns	33×	2 native slots
8	2.34 μs	41.6 ns	56×	8 native slots
32	6.85 μs	28.5 ns	240×	all-native (T=32 config)
48	8.50 μs	41.6 ns	204×[a]	32N + 16-slot trie
128	14.41 μs	32.2 ns	447×	32N + 96-slot trie

[a] Ratio drops vs. 32 slots because the mutable CLOS baseline is 46% slower at 48 slots (discontiguous memory layout), not because persistent overhead decreases.

Table 6: Modeled update cost (μs/op) at candidate thresholds T

N	T=8	T=12	T=16	T=24	T=32	T=8 strategy
8	**0.42**	0.42	0.43	0.42	0.42	all-native
12	0.75	**0.62**	0.62	0.61	0.60	8N + 4V
16	0.94	0.91	0.83	0.78	**0.77**	8N + 8V
24	**1.15**	1.23	1.28	**1.15**	**1.15**	8N + 16V
32	**1.39**	1.47	1.51	1.58	1.52	8N + 24V
48	**1.95**	1.99	2.04	2.12	2.22	8N + 40V

Bold = minimum in row. "kN + jV" = copy k native slots then batch-build j-element overflow vector. All-native = copy N slots directly. Simulated copy costs only; actual `update-slots` is 1.6–3× higher due to MOP iteration overhead (see Table 5).

4.2.1 Predicate Dispatch Micro-benchmark. We measured the per-invocation cost of a 20-method generic function, each clause specialized on a distinct Level-3 predicate, comparing FOL open and closed dispatch against standard CLOS type dispatch and a manual `cl:cond` cascade (Table 7, SBCL 2.6.0 x86-64).

Table 7: Predicate Dispatch Performance (N=20 clauses).

Dispatch Method	Time/op	Ratio (vs. CLOS)
Standard CLOS (Type)	37.1 ns	1.00×
Manual `cl:cond` Cascade	58.5 ns	1.58×
FOL `defn` (Closed Dispatch)	57.2 ns	1.54×
FOL `defgeneric` (Open Dispatch)	60.6 ns	1.63×

FOL's predicate dispatch adds ≈1.6× overhead relative to optimized CLOS type dispatch. The parity between FOL variants and the manual `cl:cond` baseline confirms that FOL's routing is as efficient as hand-written code, bounded only by the lack of $O(1)$ cacheability for Level-3 methods.

Listing 6: Naive Persistent Partition

```
(defn partition [v low high]
  (bind [pivot (get v high)]
    (loop [j low i (dec low) curr-v v]
      (if (< j high)
          (if (<= (get curr-v j) pivot)
              (bind [next-i (inc i)
                     v1 (assoc curr-v next-i (get curr-v j))
                     v2 (assoc v1 j (get curr-v next-i))]
                (recur (inc j) next-i v2))
              (recur (inc j) i curr-v))
          (bind [next-i (inc i)
                 v1 (assoc curr-v next-i (get curr-v high))
                 v2 (assoc v1 high (get curr-v next-i))]
            [v2 next-i])))))
```

Listing 7: Naive Persistent BFS Step

```
(defn bfs-step [q dists graph]
  (if (empty? q)
      dists
      (bind [u (first q)
             edges (get graph u)
             d (get dists u)]
        (loop [es edges nq (rest q) nd dists]
          (if (empty? es)
              (bfs-step nq nd graph)
              (bind [v (first es)]
                (if (= (get nd v) -1)
                    (recur (rest es)
                           (conj nq v)
                           (assoc nd v (inc d)))
                    (recur (rest es) nq nd))))))))
```

4.2.2 Lazy Evolution Micro-Benchmark. We measured single-hop (6→12 slots, two renames) and multi-hop (0→3 versions) migration against a CL baseline with a manual `update-instance-for-redefined-class :after` method. FOL's alias recovery is automatic via `:alias` annotations; CL requires a manual update per rename. FOL first-touch cost is ≈2× CL's due to functional object construction; multi-hop cost is flat (7.8–8.4 μs) as chain-replay composes alias maps in a single pass (Tables 8–9).

Table 8: Single-hop lazy migration: FOL vs. CL (μs/op, 10K instances)

Phase	FOL	CL	Notes
Migration (first touch)	14.04	7.33	one-time per dormant instance
of which: overhead vs. steady-state	10.75	7.31	SBCL migration + alias recovery
Steady-state update	3.30	0.02	post-migration, repeated use

Table 9: Multi-hop lazy migration first-touch cost (μs/op, 5K instances per cohort)

Hop depth	FOL	CL	Notes
1-hop (v2→v3)	8.13	4.10	FOL allocates new object; CL mutates in-place
2-hop (v1→v3)	8.36	5.62	extra plist scan (CL); extra hash-table pass (FOL)
3-hop (v0→v3)	7.80	5.13	both dominated by SBCL migration machinery
Steady-state	1.40	0.01	post-migration; uniform across all cohorts

4.3 Naive-Translation Workloads

Persistent *slot* updates are 33–447× slower than mutation (Table 5). Naive Quicksort and BFS (Listings 6–7) illustrate the path-copying penalty (Table 10).

Table 10: Naive-Translation Workload Results

Workload	Implementation	Time	Alloc	vs. CL
QuickSort	CL (simple-vector)	30 ms	0.8 MB	1.0×
$N = 100{,}000$	FOL Persistent (assoc)	2745 ms	2947 MB	~90×
	FOL Transient (assoc!)	1165 ms	91 MB	~39×
BFS	CL (hash-table)	8 ms	2.3 MB	1.0×
$N = 50{,}000$	FOL Persistent (assoc)	290 ms	119 MB	35×
	FOL Transient (assoc!)	52 ms	13 MB	6.5×

Quicksort overhead stems from $3.4M$ path copies; transients cut this from 90× to 39× (−97% allocation). BFS improves similarly (35× → 6.5×).

Idiomatic Reformulations. Algorithmic restructuring eliminates the translation penalty (Table 11). **Idiomatic merge sort** recurses on non-overlapping halves via `subvec` with successive `conj` calls. **BFS with lazy initialization** adds each node on first discovery, eliminating N up-front `assoc` calls.

Table 11: Idiomatic Algorithm Results: persistent and transient FOL vs. native CL

Workload	Implementation	Time	Alloc	vs. CL
Merge Sort	CL (simple-vector)	32 ms	1.6 MB	1.0×
$N = 100{,}000$	FOL persistent (`conj`)	1080 ms	623 MB	34×
	FOL transient (HAMT array)	1190 ms	81 MB	37×[a]
BFS lazy	CL (hash-table)	7 ms	2.3 MB	1.0×
$N = 50{,}000$	FOL persistent (lazy dict)	200 ms	61 MB	28×
	FOL transient (`hamt-assoc!`)	94 ms	16 MB	13×

[a] Transient is slower than persistent here: persistent `conj` touches only a root-to-leaf path of depth ≤ 5 per append, so transient setup cost is not recovered at $N = 100{,}000$.

Merge sort (34×) pays $1.7M$ $O(\log N)$ trie-node touches; this is near the theoretical minimum, as a comparison-based sort on persistent vectors requires $O(N \log^2 N)$ trie operations without transients. Transients are counterproductive (37×) because the per-`conj` path is already shallow. Lazy BFS (28×) eliminates initialization waste; transients help further (28× → 13×).

4.4 Abstract Syntax Tree Rewriting Engine

To evaluate persistent objects and predicate dispatch together, we implemented an AST optimization engine. The benchmark defines 25 persistent node classes: one abstract base `<ast-node>`, one `<op-var>` (a leaf), one `<op-lit>` (a literal with a `:val` slot), and 23 binary operator classes each with `:left` and `:right` slots. A single multi-clause `defmethod` encodes 54 algebraic rewrite rules via predicate dispatch, including nested structural patterns such as the identity-element rule:

```
(defmethod ast-optimize
  [({:keys [(left ({:keys [(val (eql 0))]} <op-lit>))
            (right r)]} <op-add>)]
  r)
```

A second 25-clause `walk` method drives recursive descent. The test tree is a 9,999-node balanced binary tree (depth 13): operator type (`mod idx 23`) cycles uniformly through all 23 binary operator classes; leaves cycle through `op-lit(0)`, `op-lit(1)`, and `op-var(:x)`. The CL baseline uses `defstruct` with a `typecase` cascade—the most favourable reasonable CL implementation.

Table 12: AST Optimizer: FOL vs. CL ($N = 100$ passes, 9,999-node balanced tree).

Phase	Implementation	Time	Alloc	vs. CL
Build	CL	0.67 ms	0.16 MB	1.0×
	FOL	51.9 ms	12.7 MB	77×
Optimizer (per pass)	CL	0.527 ms/pass	0.125 MB/pass	1.0×
	FOL	24.76 ms/pass	1.749 MB/pass	47×

FOL is 47× slower per optimizer pass (14× more memory), from three sources: **dispatch routing** (≈6×): the uniform operator distribution exercises all 25 `walk` clauses proportionally, and the $O(N)$ scan drives effective routing toward ≈6× for operators near the end of the clause list; **field reads** cost ≈50 ns vs. ≈4 ns for struct access (12×); **construction** costs ≈183 ns vs. ≈37 ns (mutable `make-instance`, 5×). The build ratio (77×) exceeds the optimizer ratio because build requires one `make-<class>` per node with no amortization. Each factor is independently measurable—routing by the dispatch micro-benchmark in Section 4.2.1 (≈1.6× vs. CLOS, corresponding to ≈6× vs. the faster `typecase` baseline); field reads and construction by the object micro-benchmarks in Section 4.2 (reads 1.4–2.1× vs. mutable CLOS, ≈12× vs. `defstruct`; construction 5× vs. mutable `make-instance`)—and their per-operation overheads, weighted by operation frequency in each pass, sum to the 47× total. In exchange, speculative rewrite passes are $O(1)$ to roll back: the modified root can be discarded without deep copying.

4.5 Logic Simulation (LSim)

LSim is an event-driven digital logic simulator that exercises persistent data structures at scale. To isolate the event-queue cost, we hold gate-connectivity constant: the **CL** baseline uses a mutable destructive leftist heap [6] ($O(1)$ allocation per merge); **FOL** uses a persistent leftist heap [19] ($O(\log n)$ copies). Table 13 covers six configurations spanning five orders of magnitude; the largest circuits used a 56 GB heap (`-dynamic-space-size 57344`). The ratio peaks at 9.4× (8x32-900) and narrows to 6.9× at the largest configuration, as the fixed $O(\log n)$ per-merge cost is amortized over more simulation work; GC contributes only 2.0% at large scale.

Table 13: LSim PQ Scaling Results (mean CPU time).

Config	Gate Evals	CL	FOL	Ratio
8bit-100	876	2.9 ms	26.5 ms	9.1×
32bit-300	10 224	27.4 ms	254 ms	9.3×
8x32-900	281 248	559 ms	5.27 s	9.4×
32x32-3000	3 850 304	8.91 s	79.1 s	8.9×
8x32x32-9000	89 708 032	126 s	903 s	7.2×
32x32x32-30000	1 183 510 528	1 990 s	13 780 s	6.9×

Normalised by gate evaluations, FOL converges toward ≈12 μs/eval while CL stays flat at 1.4–3.3 μs/eval—the narrowing ratio (9.4× to 6.9×) directly reflects this convergence as amortization reduces the FOL per-gate cost toward its floor. The mutable heap reuses freed cons cells in-place, bounding its working set to $O(\log n)$ cache-hot cells; the persistent heap allocates $O(\log n)$ fresh nodes per merge, whose non-correlated addresses increase cache pressure at small scales but are dominated by simulation work at scale.

A 2×2 ablation (Table 14) attributes overhead between the event-queue and simulation-state.

Table 14: 2×2 Ablation: Mutable vs. Persistent Heap × State Maps (mean wall time; ratios relative to CL baseline).

Circuit	CL	Hybrid-A (persist. maps)	Hybrid-B (persist. heap)	FOL
32bit-300	11.8 ms	24.1 ms (2.0×)	42.0 ms (3.6×)	118 ms (10.0×)
8x32-900	319 ms	724 ms (2.3×)	953 ms (3.0×)	2.52 s (7.9×)
32x32-3000	3.62 s	9.24 s (2.6×)	9.70 s (2.7×)	33.8 s (9.4×)
8x32x32-9000	131 s	345 s (2.6×)	246 s (1.9×)	1,024 s (7.8×)

Persistent maps add 2.0–2.6×; **persistent heap** overhead decreases from 3.6× to 1.9× as circuit scale grows. Their interaction is sub-multiplicative: allocation pressure for both overlaps in the same nursery generation. At the largest configuration (8x32x32-9000), the three independently-measured factors—persistent maps (2.6×), persistent heap (1.9×), and residual FOL overhead not attributable to either structure (1.6×, from predicate dispatch and object-creation cost)—multiply to $2.6 \times 1.9 \times 1.6 \approx 7.9\times$, matching the observed 7.8× and consistent with approximate independence at scale.

5 Threats to Validity

Single implementation: All measurements use SBCL 2.6.0 on a single platform; results may differ on other CL implementations or architectures. **Experimenter bias**: The FOL system and all CL baselines were written by the same author; baselines are idiomatic but not independently optimized, so a more aggressive CL implementation could narrow reported ratios. **Benchmark coverage**: Our evaluation covers naive-translation workloads (naive qsort, BFS) and at-scale workloads (AST Optimization, LSim). Benchmarks report means of multiple runs (small LSim configs ×20, medium

×3); the 32×32×32-30000 circuit was run once per implementation (FOL: ≈13,800 s).

The `persistent-class` metaclass relies on the MOP hooks listed in Table 1; portability to non-SBCL runtimes (e.g., CCL, ECL, ABCL) is untested.

FOL is transpiled to Common Lisp: predicate specializers require runtime evaluation and cannot be cached; type checks use an $O(N)$ linear scan rather than a discriminating function; reported dispatch ratios are therefore upper bounds for a native implementation. No static analysis exists—type errors and unreachable clauses are detected at runtime only.

Predicate side effects: Level-3 predicate specializers are arbitrary functions. The $O(N)$ linear scan evaluates predicates for all non-matching clauses; if a predicate has side effects (e.g., logging, atom updates), those effects fire for each non-matching clause before the match. Authors should ensure Level-3 predicates are pure or explicitly idempotent.

Definition-order tie-breaking is a *modularity hazard*, but its scope is strictly confined. Patterns are incomparable under $\prec$—and therefore subject to load-order sensitivity—only when every position carries the same Level-3 predicate category with no type relationship, or when patterns are structurally incomparable (map vs. sequential). For example, two methods dispatching on `[(x (prime?))]` and `[(x (fibonacci?))]` are incomparable under $\prec$; for a prime Fibonacci argument, the first-loaded method fires. Two mitigations are available within a single library: collapse overlapping Level-3 patterns into a single clause with explicit conditional logic, or demote one pattern to a type-level check (Level 2) when semantic generalization is acceptable. For *cross-library* conflicts, the recommended bridge is an explicit `:around` method: it runs before any primary method and resolves the conflict deterministically, independent of ASDF load order.

6 Related Work

Persistent data structures. FOL's trie implementation follows Bagwell's HAMT [2] and Clojure's vector and map designs [13], building on path-copying persistence theory [7]; Okasaki's purely functional heaps [19] underpin the persistent leftist heap used in LSim. The closest Common Lisp precedent is FSet [4], a functional set-theoretic collections library; FOL extends this foundation to MOP-integrated persistent *objects*, adding predicate dispatch and schema evolution—capabilities FSet does not provide.

OOP/FP bridges and pattern dispatch in other languages. Scala's case classes [18] and `match` expressions [8] provide immutable algebraic data types with structural pattern dispatch; updates use explicit `copy` methods and the class system is not MOP-integrated. Racket [10] offers `match` for structural decomposition and a separate class system [11], but the two are not unified into a single dispatch mechanism. Julia [3] provides multiple dispatch on type signatures—corresponding to FOL's Level-2 specializers—but has no predicate-level specializers and objects are mutable by default.

Predicate and multiple dispatch. Predicate dispatch was pioneered in Cecil [5] and formalized as a unified theory by Ernst et al. [9]; both require theorem-prover subsumption for method ordering. FOL sacrifices exactness for syntactic predictability. GHC's overlapping instances [12] provide a specificity ordering over type-class instance heads via `OVERLAPPING`/`OVERLAPPABLE` pragmas, though restricted to type-level patterns only (no predicate stratum), with no CLOS-style method combinations or schema evolution. Filtered dispatch [20] introduced the idea of attaching arbitrary guard predicates to methods as first-class dispatch criteria, directly motivating FOL's Level-3 predicate stratum. Dylan's singleton specializers [1] dispatch on specific object identity or value, corresponding to a restricted form of FOL's predicate specializer. Slate [21] extended multiple dispatch with prototype-based receivers; FOL instead preserves CLOS method combinations. Clojure's `defmulti` [13] dispatches on the return value of a user-supplied function; ordering is managed via `prefer-method`. Self [22]'s "slots as map entries" storage model influences FOL's object representation; FOL retains a class hierarchy and MOP-integrated dispatch rather than prototype delegation. FOL adopts Clojure's epochal time model [14], treating objects as immutable values and coordinating state change via identity primitives (Atoms, Refs).

7 Conclusion

We presented two contributions to the design of Lisp object systems. First, a **persistent metaclass system** built on the CLOS MOP: hybrid native/trie storage keeps ≈80% of classes on the 1.4–2.1× read-overhead native path; lazy alias-based schema migration delivers transparent instance evolution; and `%make-persistent` reduces object creation to 5× over mutable `make-instance`. Second, a **specificity-ordered predicate dispatch** system whose syntactic total order $\prec_{\text{disp}}$ determines a total order for all structurally or type-differentiated patterns automatically, confining load-order sensitivity to incomparable Level-3 predicates.

The benchmarks in Section 4 validate both contributions. Persistent slot updates cost 33–447× in isolation; the LSim simulation reaches 6.9× at 1.18 billion gate evaluations when only the priority queue is persistent (gate-connectivity held constant), as the fixed $O(\log n)$ per-merge cost amortizes over growing simulation work. The AST optimizer isolates the cost structure: ≈6× from dispatch routing (25-clause uniform scan), ≈12× from persistent field access, and ≈5× from persistent object construction, with $O(1)$ speculative rollback as a structural benefit.

Open problems include: native compilation to replace the $O(N)$ predicate scan with a static type-check sequence; static clause analysis to detect unreachable patterns and type errors at load time, turning the current runtime-only detection into compile-time guarantees; and propagating coordinated value transitions across a network of agents or distributed processes while preserving snapshot consistency.

FOL's implementation is available at
https://github.com/frankadrian314159/fol.

References

[1] Apple Computer, Inc. 1992. *Dylan - An Object-Oriented Dynamic Programming Language.* Apple Computer, Inc.
[2] Phil Bagwell. 2001. *Ideal Hash Trees.* Technical Report 1. EPFL, Lausanne, Switzerland.
[3] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.* 59, 1 (2017), 65–98.

[4] Scott L Burke. 2007. FSet: A functional set-theoretic collections library. https://common-lisp.net/project/fset/

[5] Craig Chambers. 1993. The Cecil Language: Design and Performance. In *Proceedings of the Conference on Object-Oriented Programming Systems, Languages, and Applications*. 243–257.

[6] Clark Allen Crane. 1972. *Linear Lists and Priority Queues as Balanced Binary Trees*. Technical Report STAN-CS-72-259. Stanford University.

[7] James R Driscoll, Neil Sarnak, Daniel D Sleator, and Robert E Tarjan. 1989. Making Data Structures Persistent. *J. Comput. System Sci.* 38, 1 (1989), 86–124.

[8] Burak Emir, Martin Odersky, and John Williams. 2007. Matching Objects with Patterns. In *ECOOP 2007–Object-Oriented Programming*. Springer, 273–298.

[9] Michael D Ernst, Craig Kaplan, and Craig Chambers. 1998. Predicate Dispatching: A Unified Theory of Dispatch. In *ECOOP'98—Object-Oriented Programming*. Springer, 186–211.

[10] Matthew Flatt et al. 2015. The Racket Manifesto. In *20 Years of the Past and 20 Years of the Future (Snapshots), Dagstuhl Seminar 15451*.

[11] Matthew Flatt, Robert Bruce Findler, and Matthias Felleisen. 2006. Scheme with Classes, Mixins, and Traits. In *Asian Symposium on Programming Languages and Systems*. Springer, 270–289.

[12] GHC Team. 2004. *GHC User's Guide: Overlapping Instances*. Glasgow Haskell Compiler. https://downloads.haskell.org/ghc/latest/docs/users_guide/.

[13] Rich Hickey. 2008. The Clojure programming language. In *Proceedings of the 2008 symposium on Dynamic languages*. 1.

[14] Rich Hickey. 2009. Are We There Yet? JVM Languages Summit Keynote. https://github.com/matthiasn/talk-transcripts/blob/master/Hickey_Rich/AreWeThereYet.md

[15] Rich Hickey. 2014. Transducers. Clojure Blog. https://clojure.org/reference/transducers

[16] Gregor Kiczales, Jim Des Rivieres, and Daniel Gureasko Bobrow. 1991. *The Art of the Metaobject Protocol*. MIT press.

[17] Michele Lanza and Radu Marinescu. 2006. *Object-Oriented Metrics in Practice*. Springer.

[18] Martin Odersky, Lex Spoon, and Bill Venners. 2016. *Programming in Scala, 3rd Edition*. Artima Press.

[19] Chris Okasaki. 1998. *Purely Functional Data Structures*. Cambridge University Press.

[20] Doug Orleans. 2002. Filtered Dispatch. In *Proceedings of the 17th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 20–26.

[21] Lee Salzman and Jonathan Aldrich. 2005. Prototypes with Multiple Dispatch: An Expressive and Dynamic Object Model. In *ECOOP 2005–Object-Oriented Programming*. Springer, 312–336.

[22] David Ungar and Randall B Smith. 1987. Self: The power of simplicity. In *ACM SIGPLAN Notices*, Vol. 22. ACM, 227–242.